

FINDINGS

Host-native code-server projects-jail + hardening — research findings

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Authority:** operator research mandate 2026-07-01 (§11.4.150 deep multi-angle research; §11.4.99 latest-source cross-reference) **Scope:** RUNTIME side of the HelixCode real-account code-server design (docs/superpowers/specs/2026-07-01-real-account-code-server-design.md + docs/superpowers/specs/2026-07-01-auth-pivot-ssh-key.md). READ + WRITE-A-DOC only. No code, no deploy, no git changes. **Runtime facts of this host (captured, §11.4.6):** ALT Linux, tcb shadow scheme, systemd --user with linger enabled for milosvasic, rootless Podman Caddy edge, code-server pinned 4.117.0, auth pivoted PAM → ssh-key challenge-response.

Honesty banner (§11.4.6). The operator has already accepted the core trade-off: **editor file-tree scoped; shell full**. This document proves *why* the file-tree scope is **cosmetic, not enforced**, exactly what IS and ISN'T enforceable, and — the load-bearing finding — that **aggressively sandboxing the code-server systemd unit is self-defeating, because the sandbox is inherited by the integrated terminal and would break the “full real-user capability” requirement**. Hardening therefore concentrates on the *auth-gate* unit and on *reachability control*, not on filesystem-jailing the editor.

Actionable hardening checklist

Ordered by security value. Each item is expanded + cited in the per-angle sections.

A. code-server launch flags (recommended)

```
code-server \  
  --auth none \  
  --bind-addr 127.0.0.1:8080 \           # OR --socket (see item C  
– stronger)  
  --user-data-dir %h/.local/share/helix-code-server \  
  --extensions-dir %h/.local/share/helix-code-server/extensions \  
  --disable-telemetry \  
  --disable-update-check \             # we pin the version; no  
phone-home  
  --disable-workspace-trust \         # single trusted  
workspace; avoids trust nag  
  --disable-file-downloads \         # optional data-egress  
reduction (see Angle 1)  
  --disable-file-uploads \           # optional  
  --ignore-last-opened \             # always reopen  
$PROJECTS_ROOT, not last dir  
"$PROJECTS_ROOT"
```

- DO isolate --user-data-dir + --extensions-dir to a HelixCode-private path so this instance does not read/write the operator's real ~/.vscode / ~/.local/share/code-server.
- DON'T rely on --disable-file-downloads/uploads or the scoped workspace as a security boundary — they are UX/egress-reduction only (Angle 1). --disable-workspace-trust "only affects the current session" per the CLI help.
- DON'T pass --skip-auth-preflight (loosens proxy auth) unless a concrete CORS need proves it.
- Caddy edge SHOULD block the code-server built-in port-proxy paths (/proxy/, /absproxy/) with a 404 if the port-forwarding feature is not used — it is the CVE-2025-47269 attack-surface class and HelixCode does not need it (Angle 2 + Angle 4).

B. helix-code-server.service (systemd --user) — MUST STAY PERMISSIVE

[Service]

```
# Resource ceilings only – cgroup-v2 + user delegation (Angle 3).  
$12.6-style.
```

```
MemoryHigh=3G
```

```
MemoryMax=4G
```

```
TasksMax=4096
```

```
# Restart backoff
```

```
Restart=on-failure
```

```
RestartSec=5s
```

```
StartLimitIntervalSec=60s
```

```
StartLimitBurst=5
```

```
# Low-risk protections that do NOT sandbox the terminal's  
*capabilities*:
```

```
ProtectClock=true
```

```
ProtectKernelTunables=true
# Everything the integrated terminal needs stays PERMISSIVE by
# omission:
# NO NoNewPrivileges (would kill setuid: sudo/su/mount/newgrp/
# passwd in the terminal)
# NO ProtectSystem=strict / PrivateUsers=true (would read-only /
# usr + nest a usersns →
# breaks installs, rootless podman, unshare, and real-uid
# tools in the terminal)
# NO ProtectHome (we REQUIRE $HOME: ~/.ssh, ~/.bashrc,
# $PROJECTS_ROOT)
# NO RestrictNamespaces (terminal tools legitimately create
# namespaces)
```

- **Load-bearing rationale:** the code-server process is the *parent* of every integrated-terminal shell. systemd sandboxing (namespaces + NoNewPrivileges) is **inherited by child processes** and cannot be relaxed by them. Any filesystem/privilege sandbox on THIS unit is therefore imposed on the user's terminal too — directly contradicting “terminal keeps full real-user capability (ssh, all host binaries).”

C. helix-auth.service (auth_gate, systemd --user) — HARDEN AGGRESSIVELY

The auth gate is a self-contained Go binary that only reads ~/.ssh/authorized_keys + shells out to ssh-keygen -Y verify. It never spawns a user login shell, so a tight sandbox costs nothing.

[Service]

```
NoNewPrivileges=true
RestrictSUIDSGID=true
# Filesystem namespacing in a --user unit REQUIRES
# PrivateUsers=true (Angle 3):
PrivateUsers=true
ProtectSystem=strict
ProtectHome=read-only # it only READS ~/.ssh/
# authorized_keys + allowed_signers
ReadWritePaths=%t/helix-auth # runtime dir for the 0600 HMAC
# secret + sockets, if any
ReadOnlyPaths=%h/.ssh
PrivateTmp=true
ProtectClock=true
ProtectKernelTunables=true
ProtectKernelModules=true
ProtectControlGroups=true
RestrictNamespaces=true
RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX
LockPersonality=true
MemoryDenyWriteExecute=true # verify ssh-keygen subprocess
# tolerates it; drop if it breaks
SystemCallArchitectures=native
# Resource + restart
```

```

MemoryMax=256M
TasksMax=256
Restart=on-failure
RestartSec=2s
StartLimitIntervalSec=60s
StartLimitBurst=10          # generous restart so the gate
                             FAILS CLOSED, not dead

```

- DO verify each namespacing line actually takes effect:
`systemctl --user show helix-auth -p NoNewPrivileges -p ProtectSystem` and `systemd-analyze --user security helix-auth`.
- DON'T set `ProtectHome=true/tmpfs` (it hides `~/.ssh` → gate can't read `authorized_keys`).
- If `MemoryDenyWriteExecute=true` breaks the `ssh-keygen` subprocess on this host, drop it (evidence-first).

D. Reachability control (close the multi-user loopback hole)

Pick ONE (in decreasing strength):

1. **Unix socket, owner-only (STRONGEST):** `code-server --socket %t/helix-code-server.sock --socket-mode 0600` + bind-mount that socket into the rootless Caddy container as the reverse-proxy upstream. Only `milosvasic` (+ root) can connect. Eliminates the loopback-TCP exposure entirely.
 2. **Loopback TCP + firewall by UID:** keep `127.0.0.1:8080` but add an `nftables/iptables OUTPUT` owner rule so only `milosvasic`'s (and the Caddy container's) UID may connect to `127.0.0.1:8080`; drop all other local UIDs.
 3. **Loopback TCP, unrestricted (WEAKEST, current design):** accept that ANY local user/process on the host can reach `127.0.0.1:8080` and get a shell **as milosvasic** — documented residual risk.
- DO keep the gate **fail-closed** (already the #1 spec invariant): auth-service outage ⇒ Caddy denies, never proxies.
 - DON'T bind `code-server` to anything but loopback/unix socket. Never `0.0.0.0`.

E. Version / update posture (Angle 4)

- 4.117.0 (2026-04-23) is patched against the only code-server-specific advisory (CVE-2025-47269, fixed 4.99.4) but is **9 releases behind** the current 4.126.0 (2026-06-24) and thus misses upstream VS Code security patches shipped 4.118→4.126. DO adopt a documented update cadence and re-pin, or consciously accept the gap per §11.4.112 with a review date.
- DO pin by exact version + verify the download checksum (§11.4.77 regen mechanism).

- `--disable-update-check` stops the 6-hourly GitHub phone-home + weekly nag.
-

Angle 1 — Editor projects-jail reality (what actually restricts navigation)

Finding: the file-tree scope is COSMETIC. It is not a filesystem boundary.

Launching code-server "\$PROJECTS_ROOT" opens \$PROJECTS_ROOT as the single-folder workspace, so the **Explorer default view** is rooted there. That is the whole of what “scoping” buys you — a default view, not a jail.

The code-server maintainer (code-asher, member) states it plainly on the exact feature request (“restrict users to only access files within the workspace directory”):

“We could restrict the file picker to the workspace root(s), and do the same with the ‘open file’ picker, but **this would not really close any security holes as the user will still have access to the files through the command line, extensions, and the debugger.** I think **the only way to reliably achieve this is to use a chroot or run code-server in a VM/container.**” — coder/code-server#6658 (duplicate of #600), accessed 2026-07-01.

The request to hard-limit filesystem access to a folder (#1251, #600, #6658) remains an **open, unimplemented feature request**. There is no CLI flag and no setting that confines the process to \$PROJECTS_ROOT.

Escape vectors that remain fully open (as the real user milosvasic)

- **Integrated terminal** — `cd /`, `cat /etc/...`, edit anything the user can (by design here).
- **File → Open Folder / Open File / Open Recent** — the picker starts anywhere the process can stat; the user can type/ navigate a parent path and open it as a new workspace.
- **Command Palette** actions, **debugger** `cwd`, **tasks**, and **extensions** — all run with the process’s full FS access.
- **Quick Open / Go to File** honors the workspace, but `files.exclude` (below) is *display* filtering, not access control.

DO (tighten the default without breaking the full terminal)

- DO pass "\$PROJECTS_ROOT" as the single workspace folder + --ignore-last-opened so a fresh window always reopens the projects root rather than the last-opened arbitrary path.
- DO set, in the HelixCode-private --user-data-dir machine/user settings:
 - "files.exclude" / "explorer.excludeGitIgnore" to hide noise (cosmetic).
 - "security.workspace.trust.enabled": false (or --disable-workspace-trust) so the trust modal doesn't block automation — noting it is NOT a boundary.
 - "window.openFoldersInNewWindow" / disable-getting-started etc. for a clean default.
- DO reduce data-egress with --disable-file-downloads (CS_DISABLE_FILE_DOWNLOADS=1) and --disable-file-uploads if download/upload via the editor is unwanted — but treat these as egress-reduction, not a jail (the terminal can still scp/curl).
- DO, if a REAL filesystem boundary is ever required, use the maintainer's answer: chroot / container / VM / bind-mount namespace — i.e. re-scope the whole runtime, not a setting.

DON'T

- DON'T represent the scoped workspace, files.exclude, or disabling openFolder as security. There is no supported way to remove the File > Open Folder command such that the terminal + extensions + debugger paths are also closed — the maintainer explicitly says a UI restriction "would not really close any security holes."
- DON'T claim the editor is "jailed to projects." The accurate claim is: **"Explorer defaults to \$PROJECTS_ROOT; the real-user shell + editor internals retain full host FS access, by operator design."**

Sources: coder/code-server#6658, #1251, #600 (feature requests, unimplemented); code-asher maintainer comment; VS Code settings/workspaces/workspace-trust docs; code-server CLI help.

Angle 2 — --auth none safety behind the Caddy gate

Is --auth none acceptable here? — Yes, CONDITIONALLY.

code-server's own guidance:

“Never expose code-server directly to the internet without some form of authentication and encryption, otherwise **someone can take over your machine via the terminal.**”
— coder guide, accessed 2026-07-01.

The guide explicitly endorses auth: none **only when a security layer sits in front** (their canonical example is SSH port-forwarding; a reverse proxy with its own auth is the same pattern): “no authentication does not pass security requirements” on its own. HelixCode's fail-closed ssh-key gate + TLS edge is a legitimate front. So --auth none is acceptable **iff** all three hold: (i) code-server is unreachable except through Caddy; (ii) the gate fails closed; (iii) TLS terminates at the edge.

The real residual risk: loopback is NOT per-user isolated

127.0.0.1:8080 on a normal host is reachable by **every local process/user in the host network namespace** — loopback has no per-UID access control. With --auth none, ANY local user who can reach that port gets an unauthenticated code-server → a terminal **as milosvasic** → full takeover of the milosvasic account, bypassing Caddy entirely. This is the one hole --auth none opens that the gate does NOT cover.

- Other **rootless containers** on the host are lower risk: rootless Podman puts each pod in its own network namespace (slirp4netns/pasta), so they cannot reach the host's 127.0.0.1 unless run with host networking or an explicit host.containers.internal route. So “other containers” ≠ automatic reach; “other host users/processes” IS automatic reach.
- **SSRF from the gate:** Caddy reverse_proxy/forward_auth to *fixed* upstreams is not SSRF. The SSRF-shaped surface is code-server's **built-in port proxy** (/proxy/<port>, /absproxy/) — the exact class of CVE-2025-47269 (session-cookie exfil via a crafted /proxy/host@evil/... URL). It is patched in 4.117.0, but the feature is unused by HelixCode and should be shut off at the edge.

DO (defense-in-depth)

- DO close the loopback hole with the strongest reachable option (checklist D): **unix socket** **--socket** ... **--socket-mode 0600** bind-mounted into Caddy is best (owner-only connect); otherwise **firewall 127.0.0.1:8080 by UID** with an nftables/iptables OUTPUT owner --uid-owner rule so only milosvasic + the Caddy container UID may connect.
- DO keep the gate **fail-closed** and TLS-terminated at Caddy (already spec'd).
- DO block /proxy/ and /absproxy/ (and /vscode-remote-resource if unused) at Caddy with a 404 — removes the CVE-2025-47269 attack-surface class even though 4.117.0 is patched.
- DO have Caddy strip inbound X-Forwarded-* from the client and set them itself, so a client cannot spoof proxy headers; forward only the minimal set code-server needs (X-Forwarded-Host/Proto).

On a “shared-secret header between Caddy and code-server”

- **Honest limitation:** with --auth none, code-server accepts *everything* and has **no hook to require a shared header** — so a Caddy-injected secret header is not validated by code-server and does NOT add a boundary on its own. A shared-secret only helps if a component that *checks* it sits in front of code-server. Two workable variants:
 1. Keep code-server on --auth password with a long random password known ONLY to Caddy, and have Caddy inject the auth cookie/credential — but this re-introduces code-server's own auth (whose cookie/proxy handling produced CVE-2025-47269), so it is weaker than the external gate.
 2. Prefer the **unix-socket / UID-firewall** boundary (a filesystem/kernel boundary), which is a real access control, over a header the backend won't enforce.
- DON'T rely on a shared header alone with --auth none.

Sources: coder/code-server guide (terminal takeover warning; auth-none-behind-proxy); GHSA-p483-wpfp-42cj / CVE-2025-47269; code-server CLI (--socket, --socket-mode, --skip-auth-preflight, --proxy-domain); iptables owner-match man page + examples; unix(7) socket permission semantics.

Angle 3 — systemd --user hardening for both units

The decisive fact: filesystem namespacing is unavailable to --user units unless PrivateUsers=true

systemd.exec(5) (Debian unstable manpage, accessed 2026-07-01) states verbatim:

“Also note that some sandboxing functionality is **generally not available in user services** (i.e. services run by the per-user service manager). Specifically, the various settings requiring **file system namespacing support (such as ProtectSystem=)** are **not available**, as the underlying kernel functionality is only accessible to privileged processes. **However, most namespacing settings, that will not work on their own in user services, will work when used in conjunction with PrivateUsers=true.**”

So in a --user unit: ProtectSystem, ProtectHome, PrivateTmp, ReadWritePaths, ReadOnlyPaths, ProtectControlGroups, RestrictNamespaces, etc. are **no-ops unless you also set PrivateUsers=true**. NoNewPrivileges and RestrictSUIDSGID (prctl/seccomp-based, not namespacing) DO work directly. Resource controls (MemoryMax, TasksMax) work on cgroup-v2 **when the controller is delegated to the user manager** (modern systemd delegates memory + pids to user@.service by default; legacy cgroup-v1 disables user resource control — verify with `systemctl --user show <u> -p MemoryMax`).

The load-bearing consequence: DON'T sandbox the code-server unit

NoNewPrivileges, user namespaces (PrivateUsers), and mount-namespaces protections are **inherited by child processes and cannot be loosened by them**. The code-server process spawns every integrated-terminal shell, so any such sandbox on helix-code-server.service is imposed on the operator's terminal too:

- NoNewPrivileges=true ⇒ setuid binaries silently fail in the terminal — sudo, su, newgrp, mount, passwd, some ping. Contradicts “all host binaries.”
- PrivateUsers=true ⇒ nested user namespace; real root maps to nobody; **rootless podman / unshare / buildah inside the terminal break** (nested users + uid-map conflicts). Contradicts “full capability.”
- ProtectSystem=strict ⇒ /usr, /etc, /boot read-only for the terminal ⇒ installs/writes fail.

- ProtectHome ⇒ hides ~/.ssh, ~/.bashrc, \$PROJECTS_ROOT ⇒ breaks the entire requirement.

⇒ For helix-code-server.service, use **resource + restart controls only**, plus at most a couple of capability-neutral protections (ProtectClock, ProtectKernelTunables). See checklist B.

For helix-auth.service — harden aggressively (it spawns no user shell)

The auth gate reads ~/.ssh/authorized_keys + runs ssh-keygen -Y verify. A tight sandbox is free here. Use checklist C:
 NoNewPrivileges, PrivateUsers=true (to unlock the namespacing set), ProtectSystem=strict, ProtectHome=read-only (must still READ ~/.ssh — do NOT use true/tmpfs), ReadOnlyPaths=%h/.ssh, ReadWritePaths=%t/helix-auth (0600 HMAC secret), PrivateTmp, RestrictSUIDSGID, RestrictNamespaces=true, RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX, LockPersonality, ProtectKernelModules/Tunables/ControlGroups, tight MemoryMax/TasksMax, generous Restart backoff (fail-closed must self-heal, not stay dead).

DO / DON'T

- DO verify each directive took effect (systemctl --user show, systemd-analyze --user security <unit>); a silently-ignored namespacing line in a user unit is a §11.4 bluff.
- DO confirm MemoryMax/TasksMax are honored (cgroup-v2 + delegation) before relying on them for §12.6.
- DON'T put NoNewPrivileges, PrivateUsers, ProtectSystem, ProtectHome, or RestrictNamespaces on the **code-server** unit — each breaks a stated terminal requirement.
- DON'T use ProtectHome=true/tmpfs on the **auth** unit (it needs to read ~/.ssh).

Sources: systemd.exec(5) Debian manpage (user-service namespacing + PrivateUsers=true); systemd.resource-control(5) (cgroup-v2 delegation, MemoryMax/TasksMax); ArchWiki systemd/Sandboxing; ageis systemd-hardening gist; NoNewPrivileges/RestrictSUIDSGID semantics.

Angle 4 — code-server install / version security notes

Version posture

- **4.117.0** released 2026-04-23; **latest is 4.126.0** (2026-06-24) — 4.117.0 is **9 releases behind**.
- The single code-server-specific advisory, **GHSA-p483-wpfp-42cj / CVE-2025-47269** (High, CVSS 8.3 — “session cookie can be extracted by having a user visit a specially crafted /proxy/ URL”), is **patched from v4.99.4 onward**, so 4.117.0 is **not vulnerable** to it. No other published code-server advisory affects 4.117.0 as of 2026-07-01.
- Residual gap: pinning 4.117.0 forgoes upstream VS Code security fixes bundled in 4.118→4.126. Each code-server release wraps a specific Code version; falling behind inherits any Code-side CVE fixed later.

DO

- DO adopt a documented update cadence (re-pin toward the latest stable, re-run the full suite) or record a §11.4.112 conscious-hold with a review date — don’t let the pin silently rot.
- DO install by **exact version + verified checksum** of the standalone tarball (code-server-4.117.0-linux-amd64.tar.gz) as the §11.4.77 regen mechanism; a user-scope `npm i -g pin` is acceptable but pin the exact version and verify the resolved binary (code-server --version) — install exit 0 ≠ correct version (§11.4.80 lesson).
- DO isolate state with `--user-data-dir + --extensions-dir` under a HelixCode-private path so this instance does not share/settle into the operator’s real `~/.local/share/code-server` or `~/.vscode` (extension & settings blast-radius isolation). Only trusted extensions in that dir.
- DO `--disable-telemetry` and `--disable-update-check` (kills the 6-hourly GitHub check + weekly nag; we pin).
- DO block the built-in proxy paths at Caddy (Angle 2) — closes the CVE-2025-47269 *class* even though patched.

DON'T

- DON'T assume “patched advisory ⇒ nothing to do” — the proxy feature remains an attack-surface class; and being 9 releases behind is its own risk.
- DON'T enable `--proxy-domain/port-proxy` unless a feature needs it.

Sources: `gh release list/view coder/code-server (4.117.0 date; 4.126.0 latest)`; GitHub Security Advisories API (GHSA-p483-wpfp-42cj / CVE-2025-47269, patched $\geq 4.99.4$); code-server CHANGELOG; code-server CLI help.

Angle 5 — Environment / .bashrc sourcing + ssh-key git from the terminal

Correcting the spec wording (`bash -l` $\Rightarrow$ `.bashrc`) — it's imprecise

Bash file-sourcing rules (bash manual, VS Code terminal docs, accessed 2026-07-01):

- An **interactive login shell** reads `~/.bash_profile` $\rightarrow$ `~/.bash_login` $\rightarrow$ `~/.profile` (first found). It does **NOT** read `~/.bashrc` directly — only if `.bash_profile/.profile` explicitly source `~/.bashrc`.
- An **interactive non-login shell** reads `~/.bashrc` directly.
- VS Code / code-server's integrated terminal, **by default on Linux, launches \$SHELL as a non-login interactive shell** — so `~/.bashrc` is **already sourced by default** without any `-l`.

$\Rightarrow$ If the operator's goal is "everything from `.bashrc` works," the **default terminal already achieves it**. Forcing `bash -l` (as the spec proposes) can actually **skip** `~/.bashrc` on a host where `~/.bash_profile` does not source it — the opposite of the intent. The correct posture depends on where the exports live:

- Exports in `~/.bashrc` $\rightarrow$ the **default (non-login) terminal already works**; do nothing, or explicitly set a non-login bash profile.
- Exports/init in `~/.profile/~/.bash_profile` (login-only) $\rightarrow$ configure a **login-shell profile** (`terminal.integrated.profiles.linux` $\rightarrow$ `{ "path": "bash", "args": ["-l"] }` + `terminal.integrated.defaultProfile.linux: "bash"`) AND ensure `~/.bash_profile` sources `~/.bashrc` (the standard convention) so BOTH are loaded.

systemd --user environment vs the interactive shell — a real pitfall

- The `systemd --user` manager has its **own environment** (from `~/.config/environment.d/*`, PAM, and `systemctl --user import-environment`), which is generally **NOT** the interactive login

shell's environment. A service started at boot via **linger** may lack variables the operator sets in the graphical/login session.

- This does **not** break the integrated terminal's env, because the terminal spawns a fresh shell that sources the rc/profile files itself — so PATH, exports, etc. are rebuilt per-terminal regardless of the service env. DO verify the code-server service's own PATH is sane if code-server itself (not the terminal) needs a tool.

ssh-key git from the terminal — SSH_AUTH_SOCKET is the trap

- The pivot's login gate reads only ~/.ssh/authorized_keys (public material). **git-over-SSH from the terminal is separate** and uses the operator's private keys in ~/.ssh.
- If the keys are **passphrase-less**, ssh/git read ~/.ssh/id_* directly — **no agent needed**, works in any terminal. This is the simplest, most reliable path and matches the spec's e2e check (git ls-remote git@github.com:vasic-digital/Code-Server.git).
- If keys are **passphrase-protected** and rely on an ssh-agent, the agent's socket is exposed via SSH_AUTH_SOCKET. A systemd --user code-server started at boot/linger will typically **NOT** inherit a desktop-session agent's SSH_AUTH_SOCKET, so the terminal won't find the agent and git may prompt/fail.
 - DO, in that case, run a **per-user ssh-agent as its own systemd --user service** and publish SSH_AUTH_SOCKET via ~/.config/environment.d/ssh_auth_socket.conf (or a fixed %t/ssh-agent.socket path), so both the manager and the terminals see a stable agent socket. GPG-agent's SSH support (gpg-agent --enable-ssh-support, SSH_AUTH_SOCKET=\$(gpgconf --list-dirs agent-ssh-socket)) is an equivalent pattern.
 - DON'T assume a login-session agent is reachable from a linger-started user service.
- DO confirm ~/.ssh perms are sane (700 dir, 600 keys, 644/600 authorized_keys) — the security suite already checks this; loose perms make ssh refuse the key AND are a leak risk (§11.4.10).

DO / DON'T summary

- DO keep the terminal as the **default non-login interactive shell** if the operator's exports live in ~/.bashrc (it already sources it) — or use bash -l + .bash_profile→.bashrc sourcing if they live in the login profile.
- DO prefer passphrase-less on-disk keys OR a systemd --user ssh-agent with a stable SSH_AUTH_SOCKET for autonomous, non-interactive ssh-key git.
- DON'T conflate the login *gate* (authorized_keys) with git-over-SSH *auth* (private keys) — they are independent.

- DON'T rely on `bash -l` to load `~/.bashrc`; verify which file the exports actually live in first.

Sources: bash manual (login vs non-login startup files); VS Code terminal profiles docs + microsoft/vscode#56061 (integrated terminal file-sourcing); systemd environment.d/import-environment; gpg-agent / ssh-agent SSH_AUTH_SOCK guidance.

Sources verified 2026-07-01

Code-server / VS Code - code-server CLI options (`--auth`, `--bind-addr`, `--socket`, `--socket-mode`, `--user-data-dir`, `--extensions-dir`, `--disable-file-downloads`, `--disable-file-uploads`, `--disable-workspace-trust`, `--disable-telemetry`, `--disable-update-check`, `--proxy-domain`, `--abs-proxy-base-path`, `--skip-auth-preflight`, `--ignore-last-opened`) — <https://github.com/coder/code-server/blob/main/src/node/cli.ts> - code-server FAQ (config, bind-addr, user-data-dir, disable-file-downloads) — <https://coder.com/docs/code-server/FAQ> - code-server “Securely Access & Expose” guide (terminal-takeover warning; auth-none-behind-proxy; reverse-proxy header forwarding) — <https://coder.com/docs/code-server/guide> and <https://github.com/coder/code-server/blob/main/docs/guide.md> - Folder-restriction feature requests + maintainer answer (cosmetic, chroot/VM/container only) — <https://github.com/coder/code-server/issues/6658> (dup of #600), <https://github.com/coder/code-server/issues/1251> - `--socket-mode` request/behavior — <https://github.com/coder/code-server/issues/1466> - Releases (4.117.0 date; 4.126.0 latest) — <https://github.com/coder/code-server/releases> ; CHANGELOG — <https://github.com/coder/code-server/blob/main/CHANGELOG.md> - Advisory GHSA-p483-wpfp-42cj / CVE-2025-47269 (proxy session-cookie exfil, patched $\geq 4.99.4$) — <https://github.com/coder/code-server/security/advisories/GHSA-p483-wpfp-42cj> , <https://nvd.nist.gov/vuln/detail/cve-2025-47269> - VS Code settings / workspaces / workspace-trust — <https://code.visualstudio.com/docs/configure/settings> , <https://code.visualstudio.com/docs/editor/workspaces> , <https://code.visualstudio.com/docs/editor/workspace-trust> - VS Code terminal profiles + integrated-terminal shell sourcing — <https://code.visualstudio.com/docs/terminal/profiles> , <https://github.com/microsoft/vscode/issues/56061>

systemd - systemd.exec(5) — user-service namespacing limitation + “will work ... with PrivateUsers=true”; NoNewPrivileges, ProtectSystem, ProtectHome,ReadWritePaths, PrivateTmp, RestrictSUIDSGID — <https://manpages.debian.org/unstable/systemd/systemd.exec.5.en.html> (freedesktop mirror 403 at fetch time) - systemd.resource-control(5) — MemoryMax/TasksMax + cgroup-v2 delegation for user services — <https://www.freedesktop.org/software/systemd/man/latest/systemd.resource-control.html> , <https://manpages.debian.org/>

testing/systemd/systemd.resource-control.5.en.html - ArchWiki
systemd/Sandboxing — <https://wiki.archlinux.org/title/Systemd/Sandboxing> (Anubis-gated at fetch; corroborated via search
excerpt: “some sandboxing functionality is generally not available
in user services”) - ageis systemd hardening options gist —
<https://gist.github.com/ageis/f5595e59b1cddb1513d1b425a323db04>

Networking / OS - iptables owner match (--uid-owner, OUTPUT-
only) to restrict loopback port by UID — <https://linux.die.net/man/8/iptables> , <https://www.cyberciti.biz/tips/block-outgoing-network-access-for-a-single-user-from-my-server-using-iptables.html> -
unix(7) socket permission semantics — <https://man7.org/linux/man-pages/man7/unix.7.html> - ssh-agent / SSH_AUTH_SOCK
guidance — https://docs.vscentrum.be/accounts/ssh_agent.html

Negative findings (documented per §11.4.99(B)): - No published
code-server advisory other than CVE-2025-47269 affects 4.117.0
(single-item advisories list, 2026-07-01). - code-server has NO
flag/setting that confines the process to \$PROJECTS_ROOT; the folder-
restriction request is unimplemented and maintainer-declared
out-of-scope for a UI fix. - A Caddy→code-server “shared-secret
header” does NOT create a boundary under --auth none, because
code-server has no mechanism to require/validate such a header.